

DDA3020 Machine Learning

Tutorial: Neural Networks

April 2026

Instructions

This tutorial covers the main ideas of neural networks, including neuron models, perceptron, multi-layer feedforward networks, backpropagation, activation functions, and deep learning concepts. Show your working clearly for all calculation questions.

Useful Formulas

For a neuron with input vector $\mathbf{x}$, weights $\mathbf{w}$, and bias b :

- Linear output: $z = \mathbf{w}^T \mathbf{x} + b$
- Sigmoid activation: $\sigma(z) = \frac{1}{1+e^{-z}}$
- ReLU activation: $\text{ReLU}(z) = \max(0, z)$
- Loss function (MSE): $L = \frac{1}{2}(y - t)^2$, where y is prediction, t is target
- Gradient descent: $\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial L}{\partial \mathbf{w}}$

Exercises

Q1. Which of the following statements about a biological neuron is correct?

- (A) Each neuron in the human brain connects to approximately 10^{11} other neurons
- (B) A neuron is activated when the number of positive electric potentials exceeds a threshold
- (C) Neurons only communicate through electrical signals, not chemical substances
- (D) The M-P neuron model was proposed in 1989

Q2. In the perceptron model, if the input is $\mathbf{x} = [1, 2]^T$, weights are $\mathbf{w} = [0.5, -1]^T$, and bias $b = 0.3$, what is the output using the sign activation function?

- (A) +1
- (B) -1
- (C) 0

(D) Cannot be determined

Q3. Which activation function is defined as $y = \max(0, z)$?

(A) Sigmoid

(B) Tanh

(C) ReLU

(D) Hard Threshold

Q4. Why can a single-layer perceptron not solve the XOR problem?

(A) XOR requires non-linear decision boundaries

(B) XOR has too many input features

(C) XOR is not a binary classification problem

(D) XOR requires continuous output values

Q5. In a multi-layer feedforward neural network, which of the following is NOT true?

(A) Connections exist only between adjacent layers

(B) Neurons in the same layer are fully connected to each other

(C) Information flows only from input layer to output layer

(D) Hidden layers transform non-linearly separable data to linearly separable

Q6. What is the primary advantage of using ReLU activation over sigmoid activation in deep networks?

(A) ReLU is differentiable everywhere

(B) ReLU helps mitigate the vanishing gradient problem

(C) ReLU produces outputs between 0 and 1

(D) ReLU is computationally more expensive

Q7. For a neuron with $z = 2.0$, compute the output using sigmoid activation function $\sigma(z) = \frac{1}{1+e^{-z}}$. (Use $e^{-2} \approx 0.135$)

Q8. For a neuron with $z = -1.0$, compute the output using ReLU activation function.

Q9. Consider a single neuron with input $x = 1.0$, weight $w = 0.5$, bias $b = 0.2$, and target $t = 1.0$. The neuron uses sigmoid activation and MSE loss $L = \frac{1}{2}(y - t)^2$.

(a) Compute the output y

(b) Compute the loss L

(c) Compute $\frac{\partial L}{\partial y}$

(d) Compute $\frac{\partial L}{\partial z}$ (where $z = w^T x + b$)

Q10. For the same neuron in Q9, compute $\frac{\partial L}{\partial w}$ and $\frac{\partial L}{\partial b}$.

Q11. If a neural network has 3 layers (input, one hidden, output), with 4 input neurons, 5 hidden neurons, and 2 output neurons, how many total weights are there? (Assume no bias terms)

Q12. In backpropagation, if $\frac{\partial L}{\partial z_j} = 0.3$ for a neuron j , and the activation function is sigmoid with output $y_j = 0.7$, what is $\frac{\partial L}{\partial y_j}$?

- Q13.** Explain the key difference between a perceptron and a multi-layer neural network.
- Q14.** Why are deep neural networks more powerful than shallow networks with the same number of parameters?
- Q15.** Describe the vanishing gradient problem and how ReLU activation helps address it.
- Q16.** What is the role of the computational graph in backpropagation?
- Q17.** Explain why a single-layer perceptron can solve the AND and OR problems but not the XOR problem.
- Q18.** In the context of neural networks, what does "universal function approximation" mean, and why is it important?
- Q19.** How does backpropagation leverage the chain rule from calculus? Provide a simple example.
- Q20.** Consider a neural network with ReLU activation trained on a binary classification task.
- (a) What happens if all weights are initialized to zero?
 - (b) Why is random initialization important?
- Q21.** A student claims: "Deeper neural networks are always better than shallow ones because they have more parameters." Do you agree? Explain your reasoning.

Solutions

Q1. Answer: B.

Explanation: Biological neurons are activated when the number of positive electric potentials exceeds a threshold, triggering chemical substances to be sent to connected neurons. Option A is incorrect (each neuron connects to $\sim 10^4$ others, not 10^{11} total neurons). Option C is incorrect (communication involves both electrical and chemical signals). Option D is incorrect (M-P model was proposed in 1943).

Q2. Answer: B.

Explanation: $z = \mathbf{w}^T \mathbf{x} + b = [0.5, -1] \cdot [1, 2] + 0.3 = 0.5 - 2 + 0.3 = -1.2$. Since $z < 0$, $\text{sign}(z) = -1$.

Q3. Answer: C.

Explanation: ReLU (Rectified Linear Unit) is defined as $y = \max(0, z)$, which outputs z when $z > 0$ and 0 otherwise.

Q4. Answer: A.

Explanation: XOR is not linearly separable, meaning no single straight line can separate the classes. A single-layer perceptron can only learn linear decision boundaries, while XOR requires non-linear boundaries that can be learned by multi-layer networks.

Q5. Answer: B.

Explanation: In feedforward networks, neurons in the same layer are NOT connected to each other. Connections exist only between adjacent layers, information flows unidirectionally, and hidden layers transform data to make it linearly separable.

Q6. Answer: B.

Explanation: ReLU helps mitigate the vanishing gradient problem because its derivative is 1 for $z > 0$, preventing gradients from becoming extremely small during backpropagation. Option A is incorrect (ReLU is not differentiable at $z = 0$). Option C is incorrect (sigmoid produces outputs between 0 and 1). Option D is incorrect (ReLU is computationally cheaper).

Q7. Answer: $y \approx 0.881$

Explanation: $\sigma(2.0) = \frac{1}{1+e^{-2}} \approx \frac{1}{1+0.135} = \frac{1}{1.135} \approx 0.881$.

Q8. Answer: $y = 0$

Explanation: $\text{ReLU}(-1.0) = \max(0, -1.0) = 0$.

Q9. Answer:

$$\begin{aligned} z &= wx + b = 0.5 \times 1.0 + 0.2 = 0.7 \\ y &= \sigma(0.7) = \frac{1}{1 + e^{-0.7}} \approx \frac{1}{1 + 0.497} \approx 0.668 \\ L &= \frac{1}{2}(0.668 - 1.0)^2 = \frac{1}{2}(-0.332)^2 \approx 0.055 \\ \frac{\partial L}{\partial y} &= y - t = 0.668 - 1.0 = -0.332 \\ \frac{\partial L}{\partial z} &= \frac{\partial L}{\partial y} \times \frac{dy}{dz} = -0.332 \times y(1 - y) = -0.332 \times 0.668 \times 0.332 \approx -0.074 \end{aligned}$$

Q10. Answer:

$$\begin{aligned} \frac{\partial L}{\partial w} &= \frac{\partial L}{\partial z} \times \frac{\partial z}{\partial w} = -0.074 \times x = -0.074 \times 1.0 = -0.074 \\ \frac{\partial L}{\partial b} &= \frac{\partial L}{\partial z} \times \frac{\partial z}{\partial b} = -0.074 \times 1 = -0.074 \end{aligned}$$

Q11. Answer: 30

Explanation: From input to hidden: 4 inputs $\times$ 5 hidden = 20 weights. From hidden to output: 5 hidden $\times$ 2 output = 10 weights. Total = 20 + 10 = 30 weights.

Q12. Answer: $\frac{\partial L}{\partial y_j} \approx 1.429$

Explanation: For sigmoid activation, $\frac{\partial L}{\partial z_j} = \frac{\partial L}{\partial y_j} \times \frac{dy_j}{dz_j} = \frac{\partial L}{\partial y_j} \times y_j(1 - y_j)$. So $\frac{\partial L}{\partial y_j} = \frac{\frac{\partial L}{\partial z_j}}{y_j(1 - y_j)} = \frac{0.3}{0.7 \times 0.3} = \frac{0.3}{0.21} \approx 1.429$.

Q13. Answer: A perceptron is a single-layer neural network with a threshold activation function that can only learn linear decision boundaries. A multi-layer neural network has one or more hidden layers with non-linear activation functions, enabling it to learn complex non-linear decision boundaries and solve problems like XOR that perceptrons cannot solve.

Q14. Answer: Deep networks can represent complex functions more efficiently than shallow networks. While a shallow network might require exponentially more neurons to represent the same function, deep networks can learn hierarchical features at different levels of abstraction. This leads to better generalization with fewer parameters and allows the network to capture intricate patterns in data.

Q15. Answer: The vanishing gradient problem occurs when gradients become extremely small during backpropagation through many layers, causing early layers to learn very slowly or not at all. This is common with sigmoid/tanh activations whose derivatives are small (≤ 0.25 for sigmoid). ReLU helps because its derivative is 1 for positive inputs, allowing gradients to flow backward without significant reduction, thus enabling effective training of deep networks.

Q16. Answer: Computational graphs provide a structured way to represent the forward computation as a directed graph, where nodes are variables/operations and edges show dependencies. During backpropagation, the graph enables efficient computation of gradients using the chain rule by traversing backward from the loss node, computing local gradients at each node and combining them. This modular approach makes it easy to implement complex neural networks.

Q17. Answer: AND and OR problems are linearly separable - a single straight line can separate the positive and negative examples. A perceptron, being a linear classifier, can learn these boundaries. XOR is not linearly separable; no single line can separate the classes. It requires a non-linear decision boundary that can only be learned by introducing hidden layers that create intermediate representations.

Q18. Answer: Universal function approximation means that a neural network with a single hidden layer (with sufficient neurons) and non-linear activation functions can approximate any continuous function to arbitrary accuracy. This is important because it theoretically guarantees that neural networks can learn extremely complex patterns and relationships in data, making them powerful universal learners for various machine learning tasks.

Q19. Answer: Backpropagation uses the chain rule to compute gradients efficiently by breaking down the derivative of the loss with respect to each parameter into a product of local derivatives. For example, in $L = \frac{1}{2}(\sigma(wx + b) - t)^2$, $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \times \frac{\partial y}{\partial z} \times \frac{\partial z}{\partial w}$, where $y = \sigma(z)$, $z = wx + b$. This allows computing gradients layer by layer from output to input.

Q20. Answer:

- (a) If all weights are initialized to zero, all neurons in each layer will compute identical outputs during forward pass, and receive identical gradients during backpropagation. This symmetry prevents the network from learning diverse features.
- (b) Random initialization breaks symmetry, allowing different neurons to learn different features. It also helps avoid local minima and ensures gradients flow properly through the network during training.

Q21. Answer: No, I do not agree. While deeper networks can represent more complex functions, they are not always better. Deeper networks can suffer from vanishing gradients, overfitting (especially with limited data), and require more computational resources. Sometimes a well-designed shallow network can outperform a deep one. The optimal depth depends on the specific task, available data, and computational constraints. Regularization techniques and proper architecture design are often more important than simply adding layers.